

# eIDPS: A Comprehensive Comparative Analysis of Packet-Level and Flow-Level Intrusion Detection and Prevention

Stamatios Kostopoulos

*Department of Electrical and Computer Engineering  
Hellenic Mediterranean University  
Herakleion, Greece  
s.kostopoulos@pasiphae.eu  
<https://orcid.org/0009-0004-8388-0134>*

Dimitra Papatsaroucha

*Department of Electrical and Computer Engineering  
Hellenic Mediterranean University  
Herakleion, Greece  
dpapatsa@hmu.gr  
<https://orcid.org/0000-0002-8894-617X>*

Ioannis Kefaloukos

*Department of Electrical and Computer Engineering  
Hellenic Mediterranean University  
Herakleion, Greece  
g.kefaloukos@pasiphae.eu  
<https://orcid.org/0000-0002-1041-9206>*

Evangelos K. Markakis

*Department of Electrical and Computer Engineering  
Hellenic Mediterranean University  
Herakleion, Greece  
emarkakis@hmu.gr  
<https://orcid.org/0000-0003-0959-598X>*

**Abstract**—Modern cybersecurity relies on Intrusion Detection and Prevention Systems (IDPS) to detect and mitigate malicious network activity. This study proposes eIDPS, a packet-level IDPS leveraging Machine Learning and the extended Berkeley Packet Filter (eBPF) for real-time threat detection within the kernel. Unlike a flow-based approach, which analyzes aggregated network flows, eIDPS operates at the packet level, enabling faster decision-making while it does not drop entire flows when a malicious packet is detected, allowing for more precise threat mitigation. Experimental evaluation using CIC-IDS2017, a benchmark intrusion detection dataset, demonstrates that eIDPS achieves higher precision and F1-score compared to a flow-based solution identified in the literature, while still operating in real time. This comparison highlights the trade-offs between packet-level and flow-based intrusion detection, offering insights into the deployment of eBPF-based security solutions.

**Index Terms**—eBPF, XDP, Neural Network, Network Intrusion Detection System, Security.

## I. INTRODUCTION

Network Intrusion Detection Systems (NIDS) [1] along with Network Intrusion Prevention Systems (NIPS) [2] are used as defense mechanisms in modern cybersecurity environments against frequent cyberattacks [3]. In addition to that, a trending technology that is combined with the aforementioned technologies is artificial intelligence (AI). NIDS and NIPS benefit from AI by being able to identify malicious patterns and anomalies inside networks [4], [5]. However, putting these state-of-the-art technologies into practice comes with a significant computational cost and challenges with scalability in big networks. Current systems frequently struggle to balance accuracy, performance, and scalability.

An emerging technology named eBPF [6] provides a sandboxed execution environment inside the Linux Kernel, allow-

ing custom security logic to be applied to network packets with minimal overhead. XDP, an extension of eBPF optimized for high-speed packet processing [7], enables packet filtering in the earliest possible state in the networking stack, before packets reach the user space.

This paper proposes eIDPS, an AI-driven Intrusion Detection and Prevention System that leverages XDP for packet-level filtering and real-time threat mitigation. The proposed solution has been evaluated against a flow-based solution identified in the literature [13] through the analysis of key metrics such as detection accuracy, system overhead, and real-time performance.

The rest of this paper is structured as follows: Section II reviews recent research on eBPF/XDP and their applications in intrusion detection and prevention; Section III details the design and implementation of eIDPS; Section IV presents the experimental evaluation and comparison with a recent flow-level proposed approach; finally, Section V summarizes the findings and discusses potential avenues for future research.

## II. STATE OF THE ART

According to recent research, using eBPF and XDP greatly improves packet processing efficiency. In contrast to alternatives such as iptables and DPDK, Jørgensen [8] claims that eBPF/XDP lowers CPU usage by reallocating CPU resources according to the packet load, rather than dedicating CPU cores exclusively to packet processing, resulting in enhanced CPU efficiency. Vieira et al. [9] and Caviglione [10] highlighted eBPF's real-time malware detection capabilities, demonstrating its potential in dynamic threat mitigation with minimal resource usage. Wang and Chang [11] showed that integrating

eBPF with NIDS enhances network throughput while reducing CPU overhead, outperforming traditional solutions, such as Snort. Despite the constraints of fixed-point arithmetic, Bachl [12] proposed an IDS and achieved 99% accuracy on CICIDS-2017 by combining eBPF with decision trees. Meanwhile, Miano et al. [14] leveraged eBPF/XDP with SmartNICs to effectively mitigate DDoS cyberattacks, showcasing its scalability in high-speed environments.

Furthermore, [15] proposed eBPF-LAIN, an eBPF and XDP-based IDS using a random forest model to detect port scanning, outperforming tools like Snort in speed and accuracy with minimal overhead. However, scalability is challenging for complex attack detection. Hassan [18] proposed iKern, an eBPF program that leverages VNF to filter packets at the kernel level, enhancing IDPS performance against volumetric and multi-vector DDoS attacks. It tackles packet loss due to buffer overflows in high-throughput networks. Nonetheless, its effectiveness on higher-speed networks and against sophisticated evasion techniques remains unaddressed. In addition to that, Nematikanti [19] proposed an IDS leveraging eBPF and ML (DT, RF, SVM, TwinSVM) for detecting DoS/DDoS attacks. The author's proposed solution preprocesses CICIDS2017 data, extracts features using ANOVA F-test, and applies ML models for intrusion detection within the kernel. The approach enhances accuracy but may face scalability challenges and real-time processing constraints while this work is yet to be peer-reviewed and published.

One of the most recent approaches proposed in the literature is that of Zhang et al. [13], who suggested an eBPF-based flow intrusion detection system that utilized neural networks and achieved strong F1 scores. However, their approach encountered difficulties in real-time processing of persistent flows, as flow-based detection methods often struggle with long-lived connections. Additionally, their system employed a non-secure method for hot updates. Hot updating is used for training the AI model and inserting the weights and bias inside the XDP program without disruptions. However, the authors assumed that the size of the source arrays structure is the same size with the destination array structure. This may lead to buffer overflow if it is not checked before updating the arrays, thus introducing potential security risks during model weight and bias updates.

Building on the reviewed literature, eIDPS introduces key improvements by leveraging packet-level detection for real-time threat detection. Unlike systems that analyze entire flows, eIDPS processes individual packets, enabling more responsive threat detection. This approach enhances detection accuracy, reduces system overhead, and improves responsiveness to cyberattacks. The main features of eIDPS include:

- **Packet-Level Detection:** Analyzes individual packets, discarding only malicious packets without dropping the entire flow.
- **Efficient for Persistent Flows:** Effectively handles long-lived connections.
- **Sanitized Hot Updates:** Employs a secure method for updating model weights and biases, mitigating potential

security risks.

### III. IMPLEMENTATION

This section details the development of the proposed solution, which leverages a neural network for network packet classification with XDP-based packet processing offloaded to the Network Interface Card (NIC). The proposed system aims to provide efficient real-time intrusion detection and prevention.

#### A. Threat Model

The proposed solution combines AI-based anomaly detection with in-kernel packet analysis, enabling packet-level threat identification and mitigation. The adversarial model assumes that attackers possess the capability to launch sophisticated network-based threats, including malicious packet injection, packet manipulation, and evasion techniques. These adversaries may have varying degrees of access, ranging from remote code execution to complete system compromise.

#### B. eIDPS Overview

eIDPS is composed of three layers: the User space, the Kernel, and the Data Link layer, as it is shown in Figure 1. The training dataset, Neural Network, and hook script are all located in the User Space. The NN utilizes the training dataset to train the AI model. The NN approach was selected because it fits better in eBPF constraints as it is easier to quantize and compress into a compact format. Furthermore, it can be easily unrolled and mapped to eBPF maps compared to decision trees or a random forest. Finally, the Hook script hooks the eBPF program into the kernel, through the hot update module. This module is employed to update the weights and bias in real-time.

Additionally, the XDP program, which functions as the Intrusion Detection/Prevention System (IDS/IPS) module and is composed of the Tracing and Security modules, is part of the Kernel Layer (version 6.1.0). These modules are responsible for analyzing packets and making decisions about accepting or rejecting IPv4 packets. The Security Module's weights and biases, which are obtained from the AI model's training through the hot update module, are quantized from floating points to integers symmetrically, by multiplying with a scaling factor and applying rounding to convert them to int32 integers. Additionally, the weights and bias serve as a guidance for this decision-making process. The Kernel IDS/IPS module is a crucial component that directly connects the eBPF program's capabilities to the broader goal of packet inspection and machine learning-based decision-making.

A crucial aspect of this approach is that the neural network's weights and biases are inserted into the Kernel using sanitized hot updating. The new weights and bias are inserted as a new input structure and are compared against the old structure to verify if their sizes match. If they differ, the weights are resized accordingly to prevent buffer overflow. The use of hot updates provides changes to the model on the fly. The retrain of the model through hot updating is crucial for implementing

new datasets in order to detect new attacks. However, this process can introduce new security risks, so the output of the model should be processed to ensure it is safe to be inserted in the XDP program. Without validation, an attacker could exploit these updates and poison the model, therefore, the hot updates should be checked and sanitized to prevent abuse. This technique ensures that updates to the model parameters occur safely and without disrupting the ongoing operation of the eIDPS. This capability allows the system to adapt to new threats by incorporating updated model parameters without requiring a kernel reboot.

Lastly, the NIC driver is located in the data link layer, which is where network traffic flows through the system. The XDP software, which runs in collaboration with the Tracing module to collect incoming and outgoing traffic for analysis, is located inside the NIC's card driver.

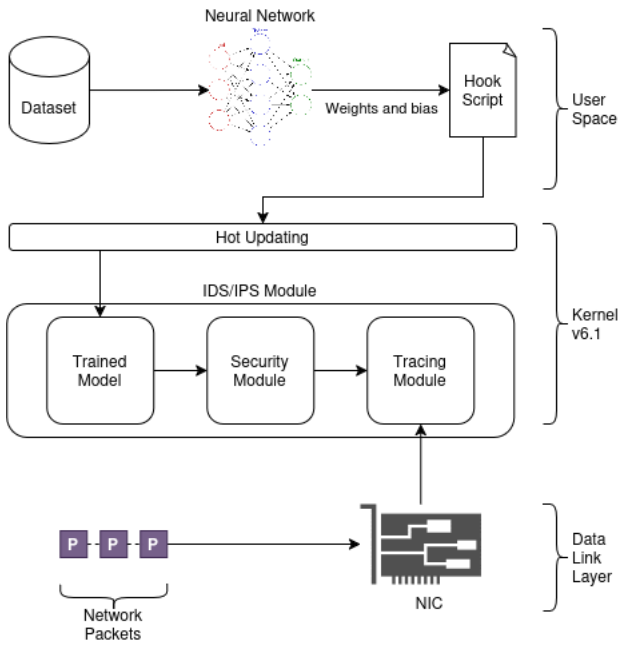


Fig. 1. eIDPS Overview

### C. Workflow

Two combined workflows are part of the suggested solution. The first illustrates the system's pre-configuration flow, while the second displays the behavior of the suggested eIDPS solution when egress or ingress traffic is recorded in the NIC.

The pre-configuration phase, depicted in Figure 2, begins with the pre-processing of the training data set, which is split into training and validation subsets. The dataset is then fed into a neural network, which serves as the core machine learning algorithm for classifying packets. Once the training is complete, the Hook Script is executed to compile and attach the eBPF program to the kernel. The trained model's initial weights and biases are then inserted into the IDS/IPS module within the kernel through the hot updating module.

The real-time monitoring workflow, illustrated in Figure 3, begins once eIDPS is deployed. As network traffic enters the



Fig. 2. Pre-configuration workflow

system via the NIC, it is intercepted by the Tracing Module. Only complete and valid IPv4 packets proceed further, ensuring that malformed or fragmented packets are discarded early to conserve resources.

Within the Security Module, the weights and biases are adjusted to comply with the floating-point limitations of eBPF. These parameters are then used to compute a Maliciousness Indication Number (MIN) for each packet, following Equation 1, adapted from [12]. The variable  $x$  represents specific features of the packet, while the weights and biases determine the significance of each feature in the classification.

The use of a linear model ensures computational efficiency and interpretability. Each packet feature is weighted and summed, with a bias value adjusting the overall threshold. The lightweight nature of this computation allows the system to process packets at high throughput with minimal latency, making it suitable for real-time network security applications.

$$y = bias + weight * x[i] \quad (1)$$

Additionally, during real-time operations, the system supports sanitized hot updating of the neural network's weights and biases. This secure mechanism allows the IDS/IPS module to receive and apply updated model parameters dynamically, ensuring that the eIDPS can continuously improve its detection capabilities without requiring a system restart. The hot update process occurs asynchronously, preventing any disruptions to ongoing packet processing.

Finally, the Security Module determines whether to allow or drop a packet based on the computed MIN value. If the MIN is greater than zero, the packet is classified as malicious and dropped. Otherwise, it is considered benign and allowed to pass through. This approach ensures that malicious traffic is efficiently mitigated while minimizing disruptions to legitimate network communications.

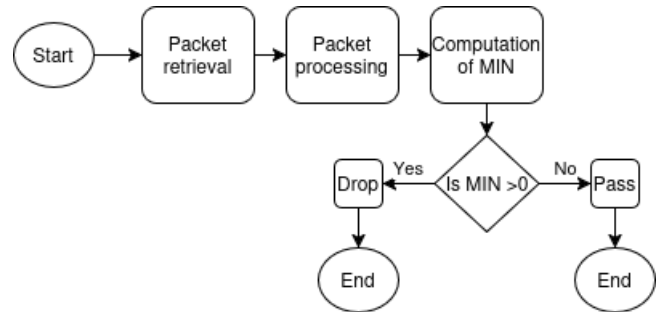


Fig. 3. eIDPS workflow

#### D. AI Model

The AI model, which is a component of the solution suggested in this paper, is implemented and trained in order to pre-configure the system at time zero, as explained in the first workflow in section III-C.

Considering that it includes 2,830,751 packets of normal and anomalous traffic, the CICIDS-2017 IDS dataset [16] was chosen for multiclass categorization. The AI model classifies individual packets in real-time, even though it is trained on network flows. Cyber attacks such as SSH brute force, port scans (like Nmap), and denial-of-service (DoS) cyberattacks (like Hulk, Slowloris, and GoldenEye) show a significant relation between packet byte values and particular packet attributes. Despite flow-based training, many cyber attacks can be identified at the packet level because they produce recurring malicious patterns, high packet rates, or noticeable anomalies in headers and payloads.

The pre-processing of the dataset included normalizing feature values, converting the "Label" column from categorical to numeric, and splitting the data into train and test, using 75% for training and 25% for testing. Before dividing, the data was shuffled using a random state function.

1) *NN configuration*: Feature importance scores in Table I were calculated using the weights of the input layer of the trained neural network. Specifically, the mean absolute weights of the input connections for each feature were computed, as these weights directly indicate the contribution of the corresponding feature to the model's predictions. Higher mean absolute weights signify greater importance. Following the methodology outlined in [17], a threshold of 0.1 was used to identify features with significant contributions. This approach ensures that only the features with meaningful predictive influence are considered, highlighting their relative impact on intrusion detection.

The hidden layer was trained using an Adam optimizer, as it can train small-scale NNs effectively and has the ability to handle sparse gradients and noisy updates. A learning rate of 0.0001 was utilized through empirical tuning and 56 neurons were used as the most important features. Furthermore, a Rectified Linear Unit (ReLU) activation function was used due to its computational efficiency in intrusion detection and it is a staple in multiclass classification. Finally, a dropout layer was added to avoid overfitting and a batch size of 500, which optimized training efficiency was used to train the model over 100 epochs.

#### IV. EVALUATION

The evaluation of the proposed solution involved a comparison between eIDPS packet-level solution with Zhang's flow-level solution [13]. Zhang's evaluation methodology focused on assessing the performance of their eBPF-based flow IDPS, which utilized neural networks. They measured the system's effectiveness using metrics such as F1-scores, precision, accuracy and recall to evaluate how well their flow-based approach identified malicious activity. Additionally, Zhang evaluated the computational overhead of their approach, assessing its

TABLE I  
FEATURE IMPORTANCE

| Feature                     | Importance |
|-----------------------------|------------|
| act_data_pkt_fwd            | 7.0        |
| Min Packet Length           | 6.8        |
| Total Fwd Packets           | 4.8        |
| Subflow Fwd Packets         | 4.5        |
| Bwd Packet Length Min       | 3.2        |
| Fwd Packet Length Min       | 2.8        |
| Down/Up Ratio               | 2.5        |
| Fwd Packet Length Max       | 2.4        |
| Fwd IAT Std                 | 2.2        |
| Fwd IAT Min                 | 2.1        |
| Avg Fwd Segment Size        | 2.0        |
| Subflow Bwd Packets         | 1.9        |
| Destination Port            | 1.8        |
| Bwd IAT Min                 | 1.6        |
| Flow IAT Std                | 1.5        |
| Flow IAT Min                | 1.4        |
| Active Max                  | 1.3        |
| Active Mean                 | 1.2        |
| Total Length of Fwd Packets | 1.1        |
| Subflow Fwd Bytes           | 1.0        |
| Bwd IAT Std                 | 0.9        |
| Active Std                  | 0.8        |
| Bwd IAT Total               | 0.7        |
| Flow IAT Mean               | 0.6        |
| Active Min                  | 0.5        |
| Bwd Packet Length Std       | 0.4        |
| Packet Length Variance      | 0.3        |
| Flow IAT Max                | 0.3        |
| Fwd IAT Mean                | 0.3        |
| Total Length of Bwd Packets | 0.2        |
| Subflow Bwd Bytes           | 0.2        |
| Bwd IAT Mean                | 0.2        |
| Init_Win_bytes_forward      | 0.2        |
| Bwd IAT Max                 | 0.2        |
| Packet Length Mean          | 0.1        |
| Fwd IAT Max                 | 0.1        |
| Fwd IAT Total               | 0.1        |
| Fwd Packet Length Std       | 0.1        |
| Packet Length Std           | 0.1        |
| Average Packet Size         | 0.1        |
| Idle Min                    | 0.1        |
| Idle Mean                   | 0.1        |
| Idle Max                    | 0.1        |
| Avg Bwd Segment Size        | 0.1        |
| Bwd Packet Length Mean      | 0.1        |
| Max Packet Length           | 0.1        |
| PSH Flag Count              | 0.1        |
| URG Flag Count              | 0.1        |
| ACK Flag Count              | 0.1        |
| Fwd PSH Flags               | 0.1        |
| Fwd URG Flags               | 0.1        |
| CWE Flag Count              | 0.1        |
| ECE Flag Count              | 0.1        |
| Flow Packets/s              | 0.1        |
| Fwd Header Length           | 0.1        |
| Bwd Header Length           | 0.1        |

impact on latency. Their evaluation also included comparisons with traditional AI algorithms to highlight improvements in detection accuracy and efficiency.

In this paper, the evaluation testbed consisted of an Intel i7-3770 processor, 32 GB RAM, a 500 GB SSD, and a NetLink BCM57781 Ethernet card supporting XDP and eBPF. To separate traffic, two Kali Linux virtual machines (VMs)

with two virtual cores and eight gigabytes of RAM were linked together via an internal network. In order to mimic six cyberattacks, SYN Scan, OS Fingerprint, Brute Force SSH, Botnet ARES, and two DoS cyberattacks (Slowloris and GoldenEye), malicious traffic was created using tools such as hping3 and Metasploit.

#### A. Experiment Scenario

The main difference between eIDPS and Zhang's solution is that the former classifies traffic on a packet level and the latter stores an entire flow and then classifies it as malicious or benign. Performance metrics of the evaluation process included CPU usage, latency, accuracy, precision, recall and F1-score. Regarding comparing the two solutions, the evaluation metrics reported in Jørgensen's work on eBPF/XDP performance optimization [8] were utilized for fair benchmarking.

#### B. AI metrics

The criteria chosen for comparing the AI models are the model's accuracy, precision, recall, and F1-score. Table II summarizes the performance metrics of the selected model, after it was tested on a testing dataset compared to Zhang's quantized model (Table V).

To be more precise, accuracy, which is the ratio of correctly predicted observations to total observations was at 98.8% while recall, which is the ratio of correctly predicted positive observations to total of observations in the actual class, was 98.8%. Furthermore, precision, which is the ratio of correctly predicted positive observations to total predicted positives, was 98.8% and F1-score, the harmonic mean of precision and recall, was at 98.8%. On the contrary, Zhang's solution resulted in 99.4% accuracy, 99.2% recall and 97.7% precision and 98.5% F1-score.

Regarding AI performance, the results show that eIDPS and Zhang's solution perform comparably. eIDPS achieves higher precision and F1 score, indicating its strength in minimizing false alarms, while Zhang's model excels in accuracy and recall, capturing a wider range of cyber attacks. This highlights a trade-off between detection coverage and false positive reduction, which is critical for real-world deployments.

TABLE II  
OVERALL METRICS

|                   | eIDPS | Zhang's solution |
|-------------------|-------|------------------|
| Overall Precision | 0.988 | 0.977            |
| Overall Recall    | 0.988 | 0.992            |
| Overall F1-score  | 0.988 | 0.985            |
| Overall Accuracy  | 0.988 | 0.994            |

#### C. CPU usage

To compare the CPU usage of both 8-core CPUs across these two solutions, the focus is on CPU overhead and its relative scaling with respect to the number of flows and packets. Specifically, Zhang's solution exhibited CPU overheads of 0.86%, 1.31%, 1.60%, and 1.64% for 1, 2, 4, and 8 flows, respectively, with a median overhead of 1.45%. Notably, when

the number of CPU cores exceeded the number of active flows, the total bandwidth declined, leading to a reduction in CPU overhead. However, the classification of flows, whether benign or malicious, remained vague, potentially impacting the accuracy of performance assessment. Furthermore, the experiment demonstrated the highest efficiency with a single flow, while efficiency decreased as the number of flows increased, reaching its lowest at 8 flows. On the other hand, eIDPS operating at the packet level, after six cyberattacks namely Slowloris, GoldenEye, Brute Force SSH, OS Fingerprint and SYN Scan resulted in 1.27%, 1.43%, 1.49%, 1.53%, 1.55%, 1.57% respectively, as it can be seen in Figure 4.

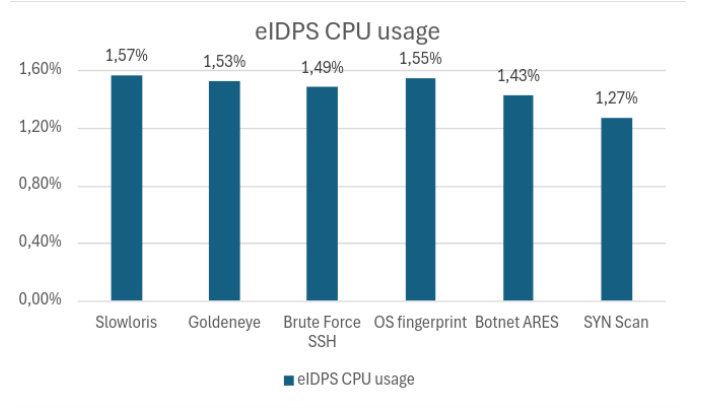


Fig. 4. eIDPS CPU Usage

To ensure a fair comparison, traffic containing both benign and malicious packets was generated for 60 seconds, mirroring the methodology used for the evaluation procedure in Zhang's research. Figure 5 presents a direct comparison of the median CPU usage between Zhang's solution and eIDPS, with a difference of 0.11%.

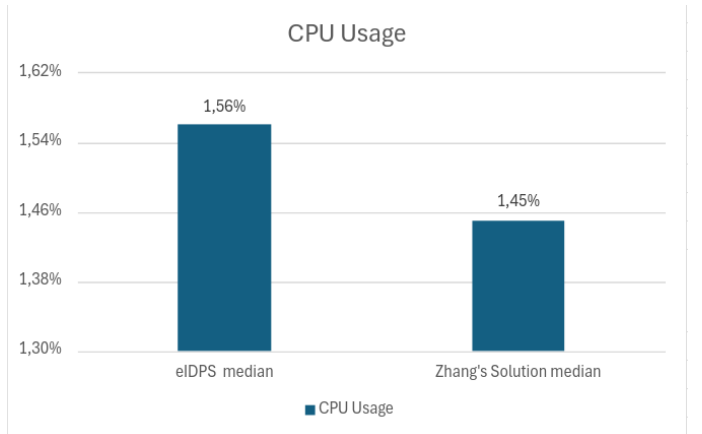


Fig. 5. median CPU Usage

Overall, both approaches exhibited similar computational efficiency. However, eIDPS offers enhanced granularity in detection and prevention while avoiding to introduce excessive computational costs, thus making it a more robust option for

environments where fine-grained traffic inspection and real-time intrusion detection are critical.

#### D. Latency

Network performance depends on latency, which is the interval of time between a packet's arrival and its filtering or forwarding. The metrics that are used for comparing eIDPS and Zhang's solution in terms of latency include the Feature Extraction per Packet (FEPP), which is the time required to extract the features from each packet, the Inference Time per Flow (IPF), which is the time needed to perform inference for a flow, the overall Processing Time, which is the total time that elapses from the moment a packet enters the system until a decision is made, and the Inference Granularity, which indicates whether the system operates at the flow level or packet level.

The expression of the Overall Processing time as a function of  $FEPP * N$  may appear that it will always return a small value for smaller FEPPs, which is reasonable. However, the intention of the comparison in this paper is to highlight the cumulative operation impact of per-packet versus per-flow inference.

TABLE III  
ZHANG'S LATENCY METRICS

| # Flows | FEPP (ns) | IPF (ns) |
|---------|-----------|----------|
| 1       | 69.50     | 5189.88  |
| 2       | 90.69     | 4570.06  |
| 4       | 90.91     | 3691.25  |
| 8       | 134.81    | 3697.88  |
| 16      | 127.90    | 2945.18  |
| 32      | 142.96    | 3333.93  |
| 64      | 145.11    | 3872.96  |
| 128     | 148.28    | 4306.88  |

FEPP in Zhang's approach ranges between 69.50ns and 148.28ns, depending on the number of concurrent flows. In contrast, eIDPS processes each packet with an average per-packet processing time of 20ns, as it operates at the packet level rather than per flow. Unlike Zhang's approach, eIDPS integrates both FEPP and inference into a single computation step, optimizing real-time packet analysis and reducing overall processing latency. The Overall Processing Time in Zhang's approach is determined by the formula:  $TotalTime = (FEPP * NumberofPackets * NumberofFlows) + (NumberofFlows * IPF)$  Table III showcases the latency values that were measured with the use of iperf by Zhang et al. Moreover, they used a total traffic of 10MB of TCP packets for each flow. One (1) TCP packet usually carries around 1.500 bytes of payload, so for 10MB of data  $10 * 1024 * 1024 / 1500 = 6990$  packets, and given  $FEPP = 145.11ns$  and  $IPF = 3872.96ns$  with 1 concurrent flow, we calculate:  $(145.11 * 1 * 6990) + (1 * 3872.96) = 1.018.191, 86ns$  Meanwhile, eIDPS processes packets directly, and achieves a total processing time of:  $6990 * 20ns = 139.800ns$  because 6990 packets are assumed to compare with the number of packets included in 1 flow in Zhang's approach. Since eIDPS operates on the packet level, it does not make sense to compare

the two solutions considering multiple flows. Thus eIDPS is compared with Zhang's solution for one flow. These results demonstrate that eIDPS has higher detection in term of speed for high packet rates and still remains real-time while doing feature extraction and inference in the same step. Table IV presents the results of the comparison of the latency metrics for Zhang's solution and eIDPS.

TABLE IV  
LATENCY METRICS

| Metric                  | Zhang's Solution | eIDPS      |
|-------------------------|------------------|------------|
| FEPP                    | 69.50ns          | 20ns       |
| Overall Processing Time | 1.018.191,86s    | 139.800ns  |
| Inference Granularity   | Per flow         | Per packet |

Both approaches store model parameters, including neural network weights and biases, using eBPF maps. The key distinction between the two is in their inference granularity:

- Zhang's method performs inference only once per completed flow, reducing computational overhead per packet, but introducing delayed attack detection, as security decisions are delayed until flow termination.
- eIDPS operates on a packet-level, allowing for immediate threat mitigation. It was worth noting that the longer processing time per packet and the latency it introduces remain negligible compared to the total processing time of the network. eIDPS is still a viable option for real-time packet filtering, even when considering its slightly higher processing latency.

#### E. Discussion

To summarize, the evaluation highlights the trade-offs between Zhang's flow-based approach and eIDPS' packet-level detection in terms of accuracy, computational efficiency, and real-time threat response.

The fundamental trade-off between flow-based and packet-based detection lies in efficiency versus detection speed. Zhang's approach minimizes the overhead per packet but delays classification until a flow is completed, making it less suitable for real-time attack prevention. Furthermore, if it detects that a flow is malicious, it drops the entire flow, even if benign packets are included. In contrast, eIDPS offers immediate response capabilities at the cost of slightly higher processing latency, which however still allows for real-time detection, ensuring continuous network monitoring.

From a deployment perspective, eIDPS is ideal for environments requiring instant threat response, such as firewalls, and cloud security. Given the increasing adoption of eBPF-based security solutions, eIDPS represents a promising direction for high-speed, real-time packet filtering and anomaly detection in modern network infrastructures. On the other hand, Zhang's solution is more suitable for post-incident forensics or scenarios where real-time mitigation is not critical. Nonetheless, a packet-based approach like eIDPS could complement Zhang's flow-based approach as the latter could prioritize the inspection

of suspicious flows, i.e. only inspecting flows if they are malicious, reducing the workload for the packet-level analyzer.

1) *Limitations*: Among the inherent limitations of the suggested solution are eBPF restrictions of one million line code limit, limitations on unbounded loops and memory relocation, as well as lack of support for floating-point arithmetic. Nevertheless, across the literature, approaches to overcome these limitations are suggested. These include techniques such as splitting an eBPF program into smaller ones that communicate via Maps (eBPF key-value stores) or tail calls (jumping between eBPF programs) to address the code lines limitation. Additionally, unrolled loops or static loop bounds can be explored to meet eBPF verifier constraints. Another common solution involves the use of global variables and pre-allocated eBPF maps for memory allocation. Finally, to emulate floating-points, integer math is used (e.g., multiply 1000 for 3 decimal places). In addition to that, eIDPS inherits limitations in detecting attacks that require correlation across multiple packets. This is a common challenge in detection systems that leverage per-packet detection, because they make decisions without considering the context of a flow. For instance, semantic attacks or evasion through fragmentation may bypass packet-level detection. In contrast flow-based approaches enable the identification of higher-level patterns and temporal relationships between the packets. This allows them to be more efficient in detecting multi-stage or stateful attacks, that span beyond the scope of a single packet.

## V. CONCLUSION

This paper proposed eIDPS, an AI-based Network Intrusion Detection and Prevention System using eBPF and XDP for efficient, real-time packet filtering. eIDPS demonstrated high accuracy and low computational overhead, outperforming Zhang's flow-level approach in terms of precision and F1-score. While it showed slightly higher latency and CPU usage, its packet-level granularity and real-time processing capabilities make it more suitable for environments requiring minimal false positives.

### A. Future Work

Future work will focus on optimizing latency and scalability, as well as on integrating advanced machine learning techniques to further improve detection. Additionally, assessing eIDPS in larger, more diverse network environments and against a broader range of attack scenarios, such as through a use case study, will help gauge its real-world effectiveness. Ultimately, the evaluation should be extended to include attack-type-specific benchmarking, to compare packet-level with flow-level detection as well as explore whether a hybrid approach could be designed in order to address the inherent limitations of eIDPS related to relationships across packets, and investigate the flexibility of eIDPS against adversarial attacks.

## ACKNOWLEDGMENT

This research initiative is supported by the European Union's Horizon Framework Programme for Research and

Innovation, under the INTACT project (Grant Agreement No. 101168438).

## REFERENCES

- [1] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *Journal of Network and Computer Applications*, vol. 36, pp. 16–24, 1 2013.
- [2] Z. Wang and X. Li, *Intrusion Prevention System Design*, 2013, pp. 375–382.
- [3] M. Albahar, "Cyber attacks and terrorism: A twenty-first century conundrum," *Science and Engineering Ethics*, vol. 25, pp. 993–1006, 82019.
- [4] Ashiku, L. and Dagli, C., 2021. Network intrusion detection system using deep learning. *Procedia Computer Science*, 185, pp.239-247.
- [5] G. Apruzzese, P. Laskov, E. M. de Oca, W. Mallouli, L. B. Rapa, A. V. Grammatopoulos, and F. D. Franco, "The role of machine learning in cybersecurity," *Digital Threats: Research and Practice*, vol. 4, pp. 1–38, 3 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3545574>
- [6] Vieira, M.A., Castanho, M.S., Pacífico, R.D., Santos, E.R., Júnior, E.P.C. and Vieira, L.F., 2020. Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Computing Surveys (CSUR)*, 53(1), pp.1-36.
- [7] N. V. Tu, J.-H. Yoo, and J. W.-K. Hong, "Building hybrid virtual network functions with express data path." *IEEE*, 10 2019, pp. 1–9.
- [8] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The express data path." *ACM*, 12 2018, pp. 54–66
- [9] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacífico, E. R. S. Santos, E. P. M. C. J. Júnior, and L. F. M. Vieira, "Fast packet processing with ebpf and xdp," *ACM Computing Surveys*, vol. 53, pp. 1–36, 1 2021
- [10] L. Caviglione, W. Mazurczyk, M. Repetto, A. Schaffhauser, and M. Zuppelli, "Kernel-level tracing for detecting stegomware and covert channels in linux environments," *Computer Networks*, vol. 191, p. 108010, 5 2021.
- [11] S.-Y. Wang and J.-C. Chang, "Design and implementation of an intrusion detection system by using extended bpf in the linux kernel," *Journal of Network and Computer Applications*, vol. 198, p. 103283, 2 2022.
- [12] M. Bachl, J. Fabini, and T. Zseby, "A flow-based ids using machine learning in ebpf," 2 2021
- [13] J. Zhang, P. Chen, Z. He, H. Chen, and X. Li, "Real-Time Intrusion Detection and Prevention with Neural Network in Kernel Using eBPF," in *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. Los Alamitos, CA, USA: IEEE Computer Society, Jun. 2024, pp. 416–428. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/DSN58291.2024.00048>
- [14] S. Miano, R. Doriguzzi-Corin, F. Risso, D. Siracusa, and R. Sommesse, "Introducing smartnics in server-based data plane processing: The ddos mitigation use case," *IEEE Access*, vol. 7, pp. 107 161–107 170, 2019.
- [15] Monteiro, J. and Sousa, B., 2024, November. eBPF Intrusion Detection System with XDP Offload support. In *2024 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)* (pp. 1-6). IEEE.]
- [16] R. Panigrahi and S. Borah, "A detailed analysis of cids2017 dataset for designing intrusion detection systems," *International Journal of Engineering & Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.
- [17] Y. Li and L. Song, "Threshold determining method for feature selection," in *2009 Second International Symposium on Electronic Commerce and Security*, vol. 2. IEEE, 2009, pp. 273–277.
- [18] Hadi, H. J., Adnan, M., Cao, Y., Hussain, F. B., Ahmad, N., Alshara, M. A., & Javed, Y. (2024). iKern: Advanced Intrusion Detection and Prevention at the Kernel Level Using eBPF. *Technologies*, 12(8), 122. <https://doi.org/10.3390/technologies12080122>
- [19] Nemalikanti A, Saifulla M A, Pavan Kumar Aakula et al. High-performance Intrusion Detection System using eBPF with Machine Learning algorithms, 05 July 2023, PREPRINT (Version 1) available at Research Square [<https://doi.org/10.21203/rs.3.rs-3140072/v1>]